

SOURCE CODE

lambda_function

```
import json
import boto3
from pprint import pprint
from parser import (
    extract_text,
    map_word_id,
    extract_table_info,
    get_key_map,
    get_value_map,
    get_kv_map,
)

def lambda_handler(event, context):
    textract = boto3.client("textract")

    if event:
        file_obj = event["Records"][0]
        bucketname = str(file_obj["s3"]["bucket"]["name"])
        filename = str(file_obj["s3"]["object"]["key"])

        print(f"Bucket: {bucketname} ::: Key: {filename}")

    response = textract.analyze_document(
        Document={
            "S3Object": {
                "Bucket": bucketname,
                "Name": filename,
```

```

        }
    },
    FeatureTypes=["FORMS", "TABLES"],
)

```

```

print(json.dumps(response))

```

```

raw_text = extract_text(response, extract_by="LINE")
word_map = map_word_id(response)
table = extract_table_info(response, word_map)
key_map = get_key_map(response, word_map)
value_map = get_value_map(response, word_map)
final_map = get_kv_map(key_map, value_map)

```

```

print(json.dumps(table))
print(json.dumps(final_map))
print(raw_text)

```

```

return {"statusCode": 200, "body": json.dumps("Thanks from Srce Cde!")}

```

parser

```

import json
import uuid

def extract_text(response, extract_by="WORD"):
    line_text = []
    for block in response["Blocks"]:
        if block["BlockType"] == extract_by:
            line_text.append(block["Text"])
    return line_text

```

```

def map_word_id(response):
    word_map = {}
    for block in response["Blocks"]:
        if block["BlockType"] == "WORD":
            word_map[block["Id"]] = block["Text"]
        if block["BlockType"] == "SELECTION_ELEMENT":
            word_map[block["Id"]] = block["SelectionStatus"]
    return word_map

def extract_table_info(response, word_map):
    row = []
    table = {}
    ri = 0
    flag = False

    for block in response["Blocks"]:
        if block["BlockType"] == "TABLE":
            key = f"table_{uuid.uuid4().hex}"
            table_n = +1
            temp_table = []

            if block["BlockType"] == "CELL":
                if block["RowIndex"] != ri:
                    flag = True
                    row = []
                    ri = block["RowIndex"]

            if "Relationships" in block:
                for relation in block["Relationships"]:
                    if relation["Type"] == "CHILD":
                        row.append(" ".join([word_map[i] for i in relation["Ids"]]))

```

```

else:

    row.append(" ")

    if flag:

        temp_table.append(row)

        table[key] = temp_table

        flag = False

    return table

def get_key_map(response, word_map):

    key_map = {}

    for block in response["Blocks"]:

        if block["BlockType"] == "KEY_VALUE_SET" and "KEY" in block["EntityTypes"]:

            for relation in block["Relationships"]:

                if relation["Type"] == "VALUE":

                    value_id = relation["Ids"]

                    if relation["Type"] == "CHILD":

                        v = " ".join([word_map[i] for i in relation["Ids"]])

                        key_map[v] = value_id

            return key_map

def get_value_map(response, word_map):

    value_map = {}

    for block in response["Blocks"]:

        if block["BlockType"] == "KEY_VALUE_SET" and "VALUE" in block["EntityTypes"]:

            if "Relationships" in block:

                for relation in block["Relationships"]:

                    if relation["Type"] == "CHILD":

                        v = " ".join([word_map[i] for i in relation["Ids"]])

                        value_map[block["Id"]] = v

            else:

```

```
value_map[block["Id"]] = "VALUE_NOT_FOUND"
```

```
return value_map
```

```
def get_kv_map(key_map, value_map):
```

```
    final_map = {}
```

```
    for i, j in key_map.items():
```

```
        final_map[i] = "".join(["".join(value_map[k]) for k in j])
```

```
    return final_map
```